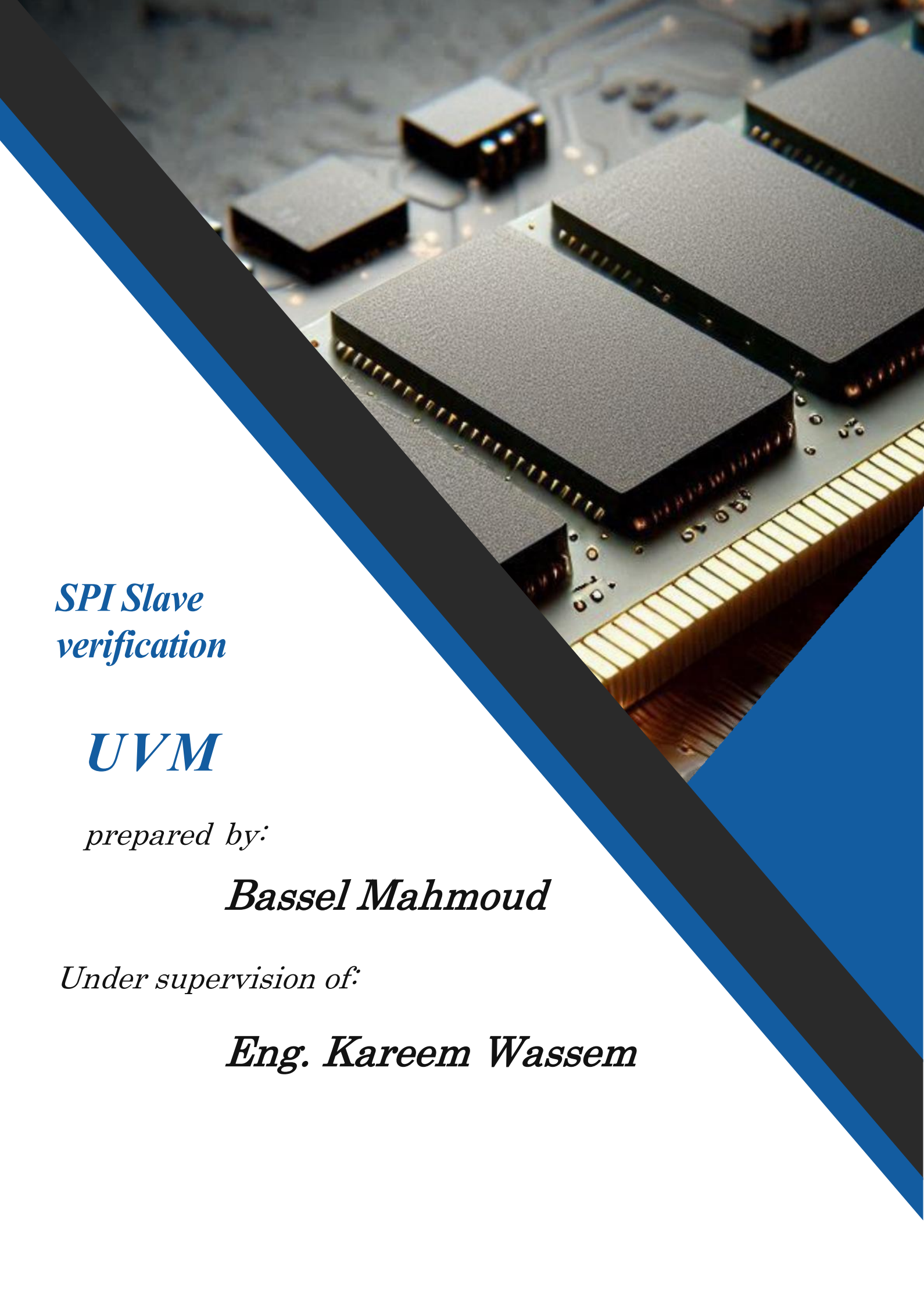


A close-up photograph of several microcontroller chips mounted on a printed circuit board (PCB). The chips are square, dark grey, and have gold-colored pins along their edges. The PCB is light grey with visible circuit traces. A diagonal blue and black graphic element runs across the right side of the image.

*SPI Slave
verification*

UVM

prepared by:

Bassel Mahmoud

Under supervision of:

Eng. Kareem Wassem

Contents

1. Interface	3
2. Shared package	3
3. Design	3
3.1. Bugs discovered	3
3.1.1. Bug 1 : READ_ADD && READ_DATA are exchanged	3
3.1.2. Bug 2 : internal counter is not reseted with rst	4
3.1.3. Bug 3 : counter counts up to 8 cycles not 9 while Read Data	4
3.2. Design code after solving the bugs	5
4. Golden Model.....	5
5. Top module.....	12
6. Test	13
7. Sequences	14
8. Sequence item.....	16
9. Config.....	19
10. Env.....	19
11. Coverage	20
12. Scoreboard	20
13. Agent	22
14. Driver	24
15. Monitor.....	24
16. Sequencer	25
17. Assertions	25
18. Do file.....	28
19. Verification Plan.....	29
20. Coverage	30



21.	<i>Questa Snaps</i>	30
-----	---------------------------	----



1. Interface

```
interface SPI_if (clk);
    input bit clk ;
    Logic MOSI, rst_n, SS_n, tx_valid;
    Logic [7:0] tx_data;
    Logic [9:0] rx_data , rx_data_gld;
    Logic rx_valid, MISO;
    Logic rx_valid_gld , MISO_gld ;
    modport DUT (
        input MOSI, clk, rst_n, SS_n, tx_valid,tx_data,
        output rx_data,rx_valid,MISO
    );
    modport gld (
        input MOSI, clk, rst_n, SS_n, tx_valid,tx_data,
        output rx_valid_gld,rx_data_gld,MISO_gld
    );
endinterface : SPI_if
```

2. Shared package

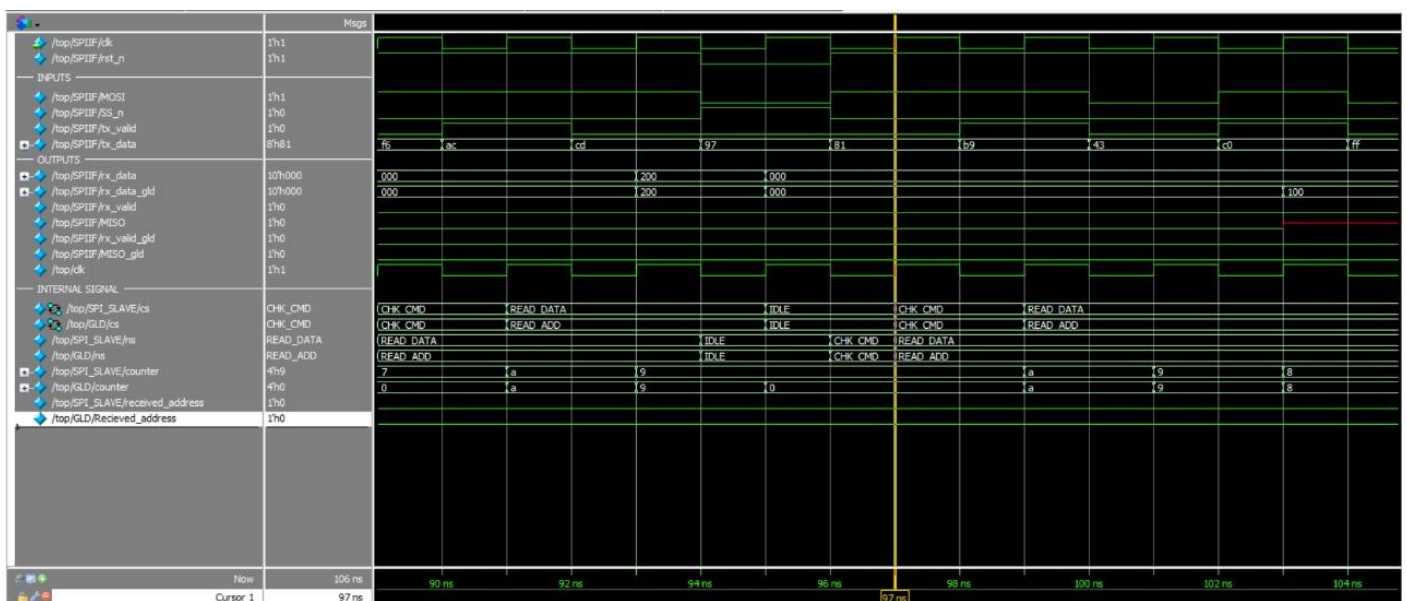
```
package SPI_shared_pkg ;
`define DEBUG
typedef enum {IDLE,WRITE,CHK_CMD,READ_ADD,READ_DATA} state_e;
typedef enum {OP_WRITE_ADDR,OP_WRITE_DATA,OP_READ_ADDR,OP_READ_DATA} op_e;
bit [10:0] MOSI_arr_shared;
endpackage
```

3. Design

3.1. Bugs discovered

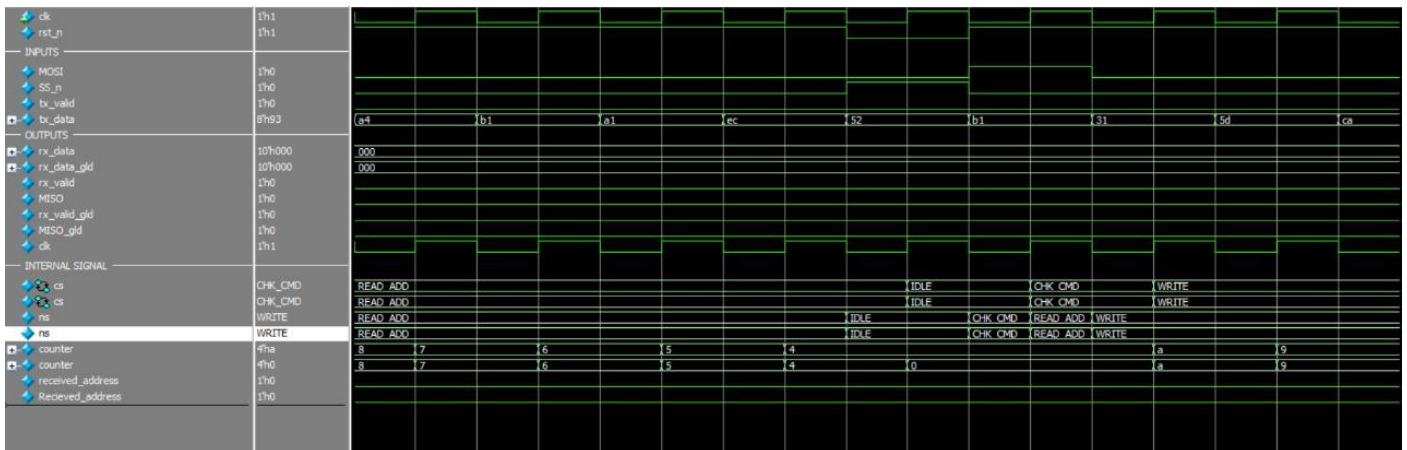
3.1.1. Bug 1 : READ_ADD && READ_DATA are exchanged

1.A) Questa sanp of the bug



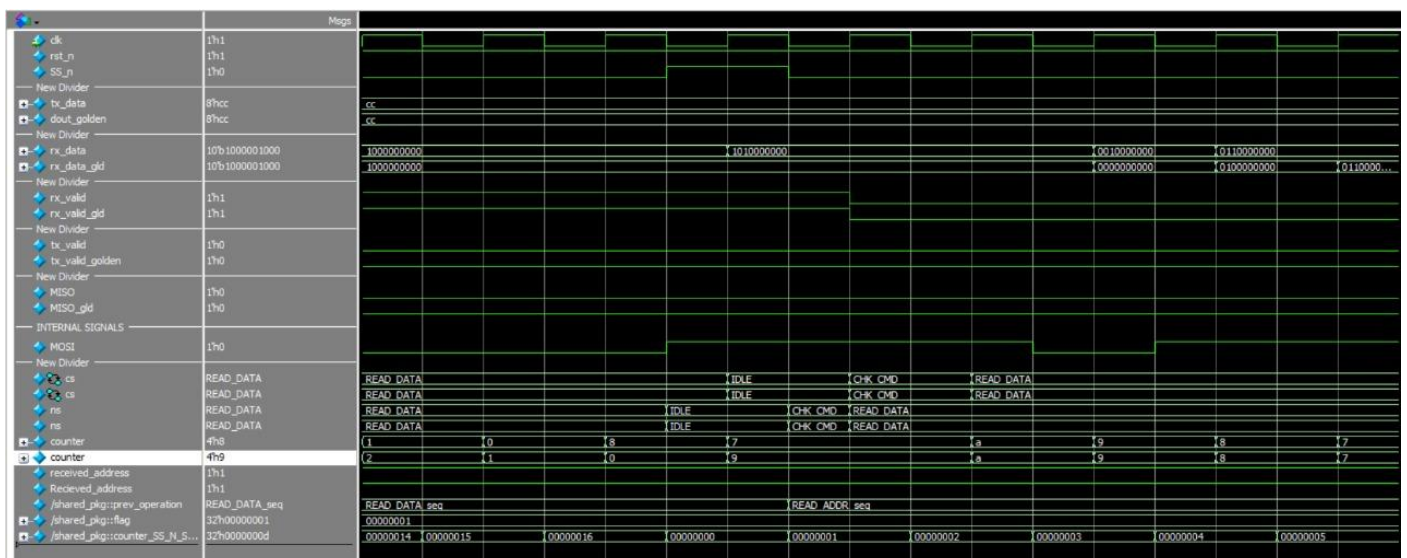
3.1.2. Bug 2 : internal counter is not reseted with rst

1.A) Questa snap of the bug



3.1.3. Bug 3 : counter counts up to 8 cycles not 9 while Read Data

1.A) Questa snap of the bug



3.2. Design code after solving the bugs

```
// module SLAVE (SPI_if.DUT SPIIF);
// import SPI_shared_pkg::*;
// // localparam IDLE      = 3'b000;
// // localparam WRITE      = 3'b001;
// // localparam CHK_CMD    = 3'b010;
// // localparam READ_ADD   = 3'b011;
// // localparam READ_DATA  = 3'b100;

// // input      MOSI, clk, rst_n, SS_n, tx_valid;
// // input      [7:0] tx_data;
// // output reg [9:0] rx_data;
// // output reg   rx_valid, MISO;
// `define DEBUG
// reg [3:0] counter;
// reg      received_address;

// state_e cs, ns;

// `ifdef DEBUG
//     op_e operation_sent ;
// `endif

// always @(posedge SPIIF.clk) begin
//     if (~SPIIF.rst_n) begin
//         cs <= IDLE;
//     end
//     else begin
//         cs <= ns;
//     end
// end

// always @(*) begin
//     case (cs)
//         IDLE : begin
//             if (SPIIF.SS_n)
//                 ns = IDLE;
//             else
//                 ns = CHK_CMD;
//         end
//         CHK_CMD : begin
//             if (SPIIF.SS_n)
//                 ns = IDLE;
//             else begin
//                 if (~SPIIF.MOSI)
//                     ns = WRITE;
//                 else begin // BUG 1 : READ_ADD && READ_DATA are exchanged
//                     if (received_address)
//                         ns = READ_DATA;
//                     else
//                         ns = READ_ADD;
//                 end
//             end
//         end
//         WRITE : begin
//             if (SPIIF.SS_n)
//                 ns = IDLE;
//             else
//                 ns = WRITE;
//         end
//         READ_ADD : begin
//             if (SPIIF.SS_n)
//                 ns = IDLE;
//             else
//                 ns = READ_ADD;
//         end
//     end
// end
```



```

//          ns = READ_ADD;
//      end
//      READ_DATA : begin
//          if (SPIIF.SS_n)
//              ns = IDLE;
//          else
//              ns = READ_DATA;
//          end
//      endcase
// end
// `ifdef DEBUG
//      always @(SPIIF.rx_data) begin
//          operation_sent = op_e'({SPIIF.rx_data[9],SPIIF.rx_data[8]});
//      end
// `endif
// //BUG2: resetting the internal counter !!
// always @(posedge SPIIF.clk) begin
//      if (~SPIIF.rst_n) begin
//          SPIIF.rx_data <= 0;
//          SPIIF.rx_valid <= 0;
//          received_address <= 0;
//          SPIIF.MISO <= 0;
//          counter <= 0 ;
//      end
//      else begin
//          case (cs)
//              IDLE : begin
//                  SPIIF.rx_valid <= 0;
//              end
//              CHK_CMD : begin
//                  counter <= 10;
//              end
//              WRITE : begin
//                  if (counter > 0) begin
//                      SPIIF.rx_data[counter-1] <= SPIIF.MOSI;
//                      counter <= counter - 1;
//                  end
//                  else begin
//                      SPIIF.rx_valid <= 1;
//                  end
//              end
//              READ_ADD : begin
//                  if (counter > 0) begin
//                      SPIIF.rx_data[counter-1] <= SPIIF.MOSI;
//                      counter <= counter - 1;
//                  end
//                  else begin
//                      SPIIF.rx_valid <= 1;
//                      received_address <= 1;
//                  end
//              end
//          end
//      end
//      READ_DATA : begin
//          if (SPIIF.tx_valid) begin
//              SPIIF.rx_valid <= 0;
//              if (counter > 0) begin
//                  SPIIF.MISO <= SPIIF.tx_data[counter-1];
//                  counter <= counter - 1;
//              end
//              else begin
//                  received_address <= 0;
//              end
//          end
//          else begin
//              if (counter > 0) begin
//                  SPIIF.rx_data[counter-1] <= SPIIF.MOSI;
//                  counter <= counter - 1;
//              end
//          end
//      end
//  end

```



```

//                end
//                else begin
//                    SPIIF.rx_valid <= 1;
//                    counter <= 8;
//                end
//            end
//        end
//    endcase
// end
// end

// endmodule

module SLAVE (SPI_if.DUT SPIIF);
import SPI_shared_pkg::*;

reg [3:0] counter;
reg      received_address;

state_e  cs, ns;

`ifdef DEBUG
    op_e operation_sent ;
`endif

always @(posedge SPIIF.clk) begin
    if (~SPIIF.rst_n) begin
        cs <= IDLE;
    end
    else begin
        cs <= ns;
    end
end

always @(*) begin
    case (cs)
        IDLE : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else
                ns = CHK_CMD;
        end
        CHK_CMD : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else begin
                if (~SPIIF.MOSI)
                    ns = WRITE;
                else begin // BUG 1 : READ_ADD && READ_DATA are exchanged
                    if (received_address)
                        ns = READ_DATA;
                    else
                        ns = READ_ADD;
                end
            end
        end
        WRITE : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else
                ns = WRITE;
        end
        READ_ADD : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else

```



```

        ns = READ_ADD;
    end
    READ_DATA : begin
        if (SPIIF.SS_n)
            ns = IDLE;
        else
            ns = READ_DATA;
        end
    endcase
end
`ifdef DEBUG
    always @(SPIIF.rx_data) begin
        operation_sent = {SPIIF.rx_data[9],SPIIF.rx_data[8]}
    end
`endif
//BUG2: resetting the internal counter !!
always @(posedge SPIIF.clk) begin
    if (~SPIIF.rst_n) begin
        SPIIF.rx_data <= 0;
        SPIIF.rx_valid <= 0;
        received_address <= 0;
        SPIIF.MISO <= 0;
        counter <= 0 ;
    end
    else begin
        case (cs)
            IDLE : begin
                SPIIF.rx_valid <= 0;
            end
            CHK_CMD : begin
                counter <= 10;
            end
            WRITE : begin
                if (counter > 0) begin
                    SPIIF.rx_data[counter-1] <= SPIIF.MOSI;
                    counter <= counter - 1;
                    SPIIF.rx_valid <= 0;
                end
                else begin
                    SPIIF.rx_valid <= 1;
                end
            end
            READ_ADD : begin
                if (counter > 0) begin
                    SPIIF.rx_data[counter-1] <= SPIIF.MOSI;
                    counter <= counter - 1;
                    SPIIF.rx_valid <= 0;
                    ///////////
                    received_address <= 0;
                end
                else begin
                    SPIIF.rx_valid <= 1;
                    received_address <= 1;
                end
            end
            READ_DATA : begin
                if (SPIIF.tx_valid) begin
                    SPIIF.rx_valid <= 0;
                    if (counter > 0) begin
                        SPIIF.MISO <= SPIIF.tx_data[counter-1];
                        counter <= counter - 1;
                    end
                    else begin
                        received_address <= 0;
                    end
                end
            end
        endcase
    end
end

```



```

        end
        else begin
            if (counter > 0) begin
                SPIIF.rx_data[counter-1] <= SPIIF.MOSI;
                counter <= counter - 1;
            end
            else begin
                SPIIF.rx_valid <= 1;
                // Bug 3 Counter should be up to 9 counts
                counter <= 8;
            end
        end
    end
end
endcase
end
end
`ifdef SIM
    // IDLE → CHK_CMD
    property a0;
    disable iff (!SPIIF.rst_n)
    (@(posedge SPIIF.clk) (cs == IDLE && (SPIIF.SS_n == 1'b0)) | => (cs == CHK_CMD));
endproperty
IDLE_CHK_CMD: assert property(a0);
cover property(a0);
    // CHK_CMD → WRITE or READ_ADD or READ_DATA
    property a1;
    disable iff (!SPIIF.rst_n)
    (@(posedge SPIIF.clk) (cs == CHK_CMD && (SPIIF.SS_n == 1'b0)) | => ((cs == WRITE) || (cs == READ_ADD) || ((cs
== READ_DATA))));
endproperty
CHK_CMD_W_WA_RD: assert property(a1);
cover property(a1);
    // WRITE → IDLE
    property a2;
    disable iff (!SPIIF.rst_n)
    (@(posedge SPIIF.clk) (cs == WRITE && (SPIIF.SS_n == 1'b1)) | => (cs == IDLE));
endproperty
WRITE_IDLE: assert property(a2);
cover property(a2);
    // READ_ADD → IDLE
    property a3;
    disable iff (!SPIIF.rst_n)
    (@(posedge SPIIF.clk) (cs == READ_ADD && (SPIIF.SS_n == 1'b1)) | => (cs == IDLE));
endproperty
READ_ADD_IDLE: assert property(a3);
cover property(a3);
    // READ_DATA → IDLE
    property a4;
    disable iff (!SPIIF.rst_n)
    (@(posedge SPIIF.clk) (cs == READ_DATA && (SPIIF.SS_n == 1'b1)) | => (cs == IDLE));
endproperty
READ_DATA_IDLE: assert property(a4);
cover property(a4);
`endif
Endmodule

```



4. Golden Model

```
module SPI_gld (SPI_if.gld SPIIF);
import SPI_shared_pkg::*;

reg [3:0] counter;
reg Recieved_address;

state_e cs, ns;

// State Memory
always @(posedge SPIIF.clk) begin
    if (~SPIIF.rst_n) begin
        cs <= IDLE;
    end
    else begin
        cs <= ns;
    end
end

// Next State Logic
always @(*) begin
    case (cs)
        IDLE : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else
                ns = CHK_CMD;
        end
        WRITE : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else
                ns = WRITE;
        end
        CHK_CMD : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else begin
                if (~SPIIF.MOSI)
                    ns = WRITE;
                else begin
                    if (Recieved_address)
                        ns = READ_DATA;
                    else
                        ns = READ_ADD;
                end
            end
        end
        READ_ADD : begin
            if (SPIIF.SS_n)
                ns = IDLE;
            else
                ns = READ_ADD;
        end
        READ_DATA : begin
```



```

        if (SPIIF.SS_n)
            ns = IDLE;
        else
            ns = READ_DATA;
        end
    endcase
end

// Ouput Logic
always @(posedge SPIIF.clk) begin
    if (~SPIIF.rst_n) begin
        SPIIF.rx_data_gld <= 0;
        SPIIF.rx_valid_gld <= 0;
        Recieved_address <= 0;
        SPIIF.MISO_gld <= 0;
        counter <= 0;
    end
    else begin
        case (cs)
            IDLE : begin
                SPIIF.rx_valid_gld <= 0;
            end
            CHK_CMD : begin
                counter <= 10;
            end
            WRITE : begin
                // Data is Ready to be Sent
                if (counter > 0) begin
                    SPIIF.rx_data_gld[counter-1] <= SPIIF.MOSI;
                    counter <= counter - 1;
                end
                else begin
                    SPIIF.rx_valid_gld <= 1;
                end
            end
            READ_ADD : begin
                // Send Read Address
                if (counter > 0) begin
                    SPIIF.rx_data_gld[counter-1] <= SPIIF.MOSI;
                    counter <= counter - 1;
                end
                else begin
                    SPIIF.rx_valid_gld <= 1;
                    // Next Time a Read Command will be Sent
                    Recieved_address <= 1;
                end
            end
            READ_DATA : begin
                if (SPIIF.tx_valid) begin
                    SPIIF.rx_valid_gld <= 0;
                    // Read Data
                    if (counter == 0)
                        Recieved_address <= 0;
                    else begin
                        SPIIF.MISO_gld <= SPIIF.tx_data[counter-1];
                        counter <= counter - 1;
                    end
                end
            end
        end
    end
end

```



```

        // Ask to read the Data
    else begin
        // Send Data Read Request to RAM
        if (counter > 0) begin
            SPIIF.rx_data_gld[counter-1] <= SPIIF.MOSI;
            counter <= counter - 1;
        end
        // Read the Data From the RAM
        else begin
            SPIIF.rx_valid_gld <= 1;
            // It Should be 8 but we have one clock cycle to wait SPIIF.tx_valid = 1
            after rx_valid = 1 by 1 clk cycle
                counter <= 9;
        end
    end
end
endcase
end
end
endmodule

```

5. Top module

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import SPI_test_pkg::*;

module top();
    // Clock generation
    bit clk ;
    initial begin
        clk = 0 ;
        forever begin
            #1; clk = ~clk ;
        end
    end
    // Instantiate the interface and DUT
    SPI_if SPIIF (clk);
    SLAVE SPI_SLAVE (SPIIF);
    SPI_gld GLD (SPIIF);

    // Bind assertions module to SPI design unit (adjust instance path as needed)
    bind SPI_SLAVE SPI_assertions SPI_ASSERT (SPIIF);

    // run test using run_test task
    initial begin
        uvm_config_db#(virtual SPI_if)::set(null, "uvm_test_top", "SPI_IF", SPIIF);
        run_test("SPI_test");
    end
endmodule

```



6. Test

```
package SPI_test_pkg;
import SPI_env_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
import SPI_config_pkg::*;
import SPI_shared_pkg::*;
import SPI_sequence_item_pkg::*;
import SPI_sequence_pkg::*;
import SPI_config_pkg::*;
import SPI_env_pkg::*;

class SPI_test extends uvm_test;
    `uvm_component_utils(SPI_test)
    SPI_env env ;
    SPI_config SPI_cfg;
    SPI_main_sequence main_seq ;
    SPI_corner_sequence corner_seq ;
    SPI_reset_sequence reset_seq ;
    virtual SPI_if SPI_vif_cfg;

    function new(string name = "SPI_test",uvm_component parent = null);
        super.new(name,parent);
    endfunction
    function void build_phase (uvm_phase phase);
        super.build_phase(phase);
        env = SPI_env::type_id::create("env",this);
        SPI_cfg = SPI_config::type_id::create("SPI_cfg");
        main_seq = SPI_main_sequence::type_id::create("main_seq");
        corner_seq = SPI_corner_sequence::type_id::create("corner_seq");
        reset_seq = SPI_reset_sequence::type_id::create("reset_seq");
        if(!uvm_config_db #(virtual SPI_if)::get(this,"","SPI_IF",SPI_cfg.SPI_vif_cfg))
            `uvm_fatal("build_phase","Test - unable to get the virtual interface")
        uvm_config_db #(SPI_config)::set(this,"*", "SPI_CFG",SPI_cfg);
        SPI_cfg.is_active = UVM_ACTIVE;
    endfunction

    task run_phase (uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);
        `uvm_info("run_phase","RESET_SEQ",UVM_MEDIUM)
        reset_seq.start(env.agent.sqr);
        `uvm_info("run_phase","CORNER_SEQ",UVM_MEDIUM)
        corner_seq.start(env.agent.sqr);
        `uvm_info("run_phase","MAIN_SEQ",UVM_MEDIUM)
        main_seq.start(env.agent.sqr);
        phase.drop_objection(this);
    endtask: run_phase
endclass: SPI_test
endpackage
```



7. Sequences

```
package SPI_sequence_pkg;
import uvm_pkg::*;
import SPI_shared_pkg::*;
import SPI_sequence_item_pkg::*;
`include "uvm_macros.svh"

class SPI_main_sequence extends uvm_sequence #(SPI_sequence_item);
    `uvm_object_utils(SPI_main_sequence)

    SPI_sequence_item seq_item ;

    function new(string name = "SPI_main_sequence");
        super.new(name);
    endfunction //new()

    task body ();
        seq_item = SPI_sequence_item::type_id::create("seq_item");
        repeat(50000) begin
            start_item(seq_item);
            assert (seq_item.randomize());
            finish_item(seq_item);
        end
        repeat(1000)begin
            seq_item = SPI_sequence_item::type_id::create("seq_item");
            start_item(seq_item);
            seq_item.MOSI_arr.rand_mode(1);
            seq_item.rst_n.rand_mode(1);
            seq_item.tx_data.rand_mode(1);
            seq_item.c_rst_n.constraint_mode(0);
            seq_item.c_MOSI.constraint_mode(0);
            assert (seq_item.randomize());
            finish_item(seq_item);
        end
    endtask
endclass //SPI_main_sequence extends uvm_sequence

class SPI_corner_sequence extends uvm_sequence #(SPI_sequence_item);
    `uvm_object_utils(SPI_corner_sequence)

    SPI_sequence_item seq_item ;

    function new(string name = "SPI_corner_sequence");
        super.new(name);
    endfunction //new()

    task body ();
        seq_item = SPI_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 1;
        finish_item(seq_item);
        seq_item = SPI_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0;
        seq_item.SS_n = 0 ;
        finish_item(seq_item);
```



```

        seq_item = SPI_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.SS_n = 1 ;
        finish_item(seq_item);
        seq_item = SPI_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.SS_n = 0;
        finish_item(seq_item);
    endtask
endclass

class SPI_reset_sequence extends uvm_sequence #(SPI_sequence_item);
    `uvm_object_utils(SPI_reset_sequence)

    SPI_sequence_item seq_item ;

    function new(string name = "SPI_reset_sequence");
        super.new(name);
    endfunction //new()

    task body ();
        // First transaction - assert reset
        seq_item = SPI_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 1;
        finish_item(seq_item);

        // Second transaction - deassert reset
        seq_item = SPI_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0;
        finish_item(seq_item);

        // Third transaction - assert reset again
        seq_item = SPI_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 1;
        finish_item(seq_item);
    endtask
endclass //SPI_reset_sequence extends uvm_sequence

endpackage

```



8. Sequence item

```
package SPI_sequence_item_pkg;
import uvm_pkg::*;
import SPI_shared_pkg::*;
`include "uvm_macros.svh"
`define WRITE_ADDR_ins 3'b000
`define WRITE_DATA_ins 3'b001
`define READ_ADDR_ins 3'b110
`define READ_DATA_ins 3'b111

class SPI_sequence_item extends uvm_sequence_item;
    `uvm_object_utils(SPI_sequence_item)

    rand bit rst_n;
    rand bit [7:0] tx_data;
    rand bit MOSI_arr [11];

    // Non-randomized signals
    bit SS_n, tx_valid;
    logic MOSI;
    logic rx_valid, MISO;
    logic rx_valid_gld, MISO_gld;
    logic [9:0] rx_data, rx_data_gld;

    // Control variables
    int counter_SS_n;
    bit f_read_data;
    bit f_read_addr;
    int MOSI_arr_index;
    bit [2:0] cmd;

    function new(string name = "SPI_sequence_item");
        super.new(name);
        counter_SS_n = 0;
        MOSI_arr_index = 0;
        SS_n = 1;
        tx_valid = 0;
        f_read_data = 0;
        f_read_addr = 0;
    endfunction

    // Constraint 1: Reset deasserted most of the time (90%)
    constraint c_rst_n {
        rst_n dist {0:/10, 1:/90};
        // if (SS_n == 0){
        //     rst_n == 1 ;
        // }
    }

    // Constraint 3: First 3 bits of MOSI_arr must be valid command
    constraint c_MOSI {
        {MOSI_arr[0], MOSI_arr[1], MOSI_arr[2]} inside {3'b000, 3'b001, 3'b110, 3'b111};
    }

    // Post-randomize handles constraints 2, 3, and 4
```



```

function void post_randomize();

// Handle reset
if (~rst_n) begin
    counter_SS_n = 0;
    MOSI_arr_index = 0;
    SS_n = 1;
    tx_valid = 0;
    f_read_data = 0;
    f_read_addr = 0;
    MOSI = 0;
    return;
end

cmd = {MOSI_arr[0], MOSI_arr[1], MOSI_arr[2]};

// Update flags based on command
if (cmd == `READ_ADDR_ins) begin
    f_read_addr = 1;
    f_read_data = 0;
end
else if (cmd == `READ_DATA_ins && f_read_addr) begin
    f_read_addr = 0;
    f_read_data = 1;
end
else if (cmd == `WRITE_ADDR_ins || cmd == `WRITE_DATA_ins) begin
    f_read_addr = 0;
    f_read_data = 0;
end
if (f_read_data) begin
    if (counter_SS_n >= 22) begin
        MOSI_arr.rand_mode(1);
        rst_n.rand_mode(1);
        tx_data.rand_mode(1);
        c_rst_n.constraint_mode(1);
        SS_n = 1;
        counter_SS_n = 0;
        MOSI_arr_index = 0;
    end
    else begin
        MOSI_arr.rand_mode(0);
        rst_n.rand_mode(0);
        tx_data.rand_mode(0);
        c_rst_n.constraint_mode(0);

        SS_n = 0;
        counter_SS_n++;
        MOSI_arr_index++;
    end
end
else begin
    if (counter_SS_n >= 12) begin
        MOSI_arr.rand_mode(1);
        rst_n.rand_mode(1);
        tx_data.rand_mode(1);
        c_rst_n.constraint_mode(1);
        SS_n = 1;
    end
    else begin
        MOSI_arr.rand_mode(0);
        rst_n.rand_mode(0);
        tx_data.rand_mode(0);
        c_rst_n.constraint_mode(0);
        SS_n = 0;
        counter_SS_n++;
        MOSI_arr_index++;
    end
end
end

```



```

        counter_SS_n = 0;
        MOSI_arr_index = 0;
    end
    else begin
        MOSI_arr.rand_mode(0);
        rst_n.rand_mode(0);
        tx_data.rand_mode(0);
        c_rst_n.constraint_mode(0);
        SS_n = 0;
        counter_SS_n++;
        MOSI_arr_index++;
    end
end

if (MOSI_arr_index < 11)
    MOSI = MOSI_arr[MOSI_arr_index];
else
    MOSI = 0;

tx_valid = f_read_data;

// MOSI_arr_shared = MOSI_arr;

endfunction

function string convert2string();
    string s;
    string cmd_str;
    case ({MOSI_arr[0], MOSI_arr[1], MOSI_arr[2]})
        `WRITE_ADDR_ins: cmd_str = "WRITE_ADDR";
        `WRITE_DATA_ins: cmd_str = "WRITE_DATA";
        `READ_ADDR_ins:  cmd_str = "READ_ADDR";
        `READ_DATA_ins:  cmd_str = "READ_DATA";
        default: cmd_str = "UNKNOWN";
    endcase
    s = "\n----- SLAVE Sequence Item ----- \n";
    s = {s, $sformatf("MOSI           : %0b\n", MOSI)};
    s = {s, $sformatf("rst_n          : %0b\n", rst_n)};
    s = {s, $sformatf("SS_n           : %0b\n", SS_n)};
    s = {s, $sformatf("tx_valid       : %0b\n", tx_valid)};
    s = {s, $sformatf("tx_data        : 0x%0h\n", tx_data)};
    s = {s, $sformatf("rx_data        : 0x%0h\n", rx_data)};
    s = {s, $sformatf("rx_data_golden : 0x%0h\n", rx_data_gld)};
    s = {s, $sformatf("rx_valid       : %0b\n", rx_valid)};
    s = {s, $sformatf("rx_valid_golden : %0b\n", rx_valid_gld)};
    s = {s, $sformatf("MISO           : %0b\n", MISO)};
    s = {s, $sformatf("MISO_golden    : %0b\n", MISO_gld)};
    s = {s, $sformatf("command        : %s\n", cmd_str)};
    s = {s, $sformatf("f_read_data    : %0b\n", f_read_data)};
    s = {s, $sformatf("f_read_addr    : %0b\n", f_read_addr)};
    s = {s, $sformatf("counter_SS_n   : %0d\n", counter_SS_n)};
    s = {s, $sformatf("MOSI_arr_index : %0d\n", MOSI_arr_index)};
    s = {s, "----- \n"};
    return s;
endfunction

```



```

function string convert2string_stimulus();
    string cmd_str;
    case ({MOSI_arr[0], MOSI_arr[1], MOSI_arr[2]})
        `WRITE_ADDR_ins: cmd_str = "WRITE_ADDR";
        `WRITE_DATA_ins: cmd_str = "WRITE_DATA";
        `READ_ADDR_ins: cmd_str = "READ_ADDR";
        `READ_DATA_ins: cmd_str = "READ_DATA";
        default: cmd_str = "UNKNOWN";
    endcase
    return $sformatf("@%0t: rst_n=%b MOSI=%b SS_n=%b tx_valid=%b tx_data=%0h cmd=%s
f_read_data=%b f_read_addr=%b counter_SS_n=%0d MOSI_arr_index=%0d",
                    $time, rst_n, MOSI, SS_n, tx_valid, tx_data, cmd_str, f_read_data,
f_read_addr, counter_SS_n, MOSI_arr_index);
endfunction
endclass
endpackage

```

9. Config

```

package SPI_config_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
class SPI_config extends uvm_object;
    `uvm_object_utils(SPI_config)

    virtual SPI_if SPI_vif_cfg;

    uvm_active_passive_enum is_active;

    function new (string name = "SPI_config");
        super.new(name);
    endfunction
endclass
endpackage

```

10. Env

```

package SPI_env_pkg;
import uvm_pkg::*;

`include "uvm_macros.svh"
import SPI_shared_pkg::*;
import SPI_sequence_item_pkg::*;
import SPI_coverage_pkg::*;
import SPI_agent_pkg::*;
import SPI_scoreboard_pkg::*;

class SPI_env extends uvm_env;
    `uvm_component_utils(SPI_env)
    SPI_coverage cov ;
    SPI_scoreboard sb ;
    SPI_agent agent ;

    function new (string name = "SPI_env", uvm_component parent = null);
        super.new(name, parent);
    endfunction
    function void build_phase (uvm_phase phase);
        super.build_phase(phase);
        cov = SPI_coverage::type_id::create("cov", this);
        sb = SPI_scoreboard::type_id::create("sb", this);
    endfunction
endclass
endpackage

```

```

    agent = SPI_agent::type_id::create("agent",this);
endfunction
function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    agent.agt_ap.connect(cov.cov_export);
    agent.agt_ap.connect(sb.sb_export);
endfunction

endclass
endpackage

```

11. Coverage

```

package SPI_coverage_pkg;
import uvm_pkg::*;
import SPI_shared_pkg::*;
import SPI_sequence_item_pkg::*;
`include "uvm_macros.svh"
class SPI_coverage extends uvm_component;
    `uvm_component_utils(SPI_coverage)

    uvm_analysis_export #(SPI_sequence_item) cov_export;
    uvm_tlm_analysis_fifo #(SPI_sequence_item) cov_fifo;
    SPI_sequence_item cov_seq_item;
    covergroup cvr_gp ;
        rx_data_msb_cp : coverpoint cov_seq_item.rx_data[9:8] {
            bins msb_00 = {2'b00};
            bins msb_01 = {2'b01};
            bins msb_10 = {2'b10};
            bins msb_11 = {2'b11};
            bins msb_transitions = (2'b00 => 2'b01), (2'b00 => 2'b10), (2'b00 => 2'b11),
                                   (2'b01 => 2'b00), (2'b01 => 2'b10), (2'b01 => 2'b11),
                                   (2'b10 => 2'b00), (2'b10 => 2'b01), (2'b10 => 2'b11),
                                   (2'b11 => 2'b00), (2'b11 => 2'b01), (2'b11 => 2'b10);
        }
        SS_n_cp : coverpoint cov_seq_item.SS_n {
            bins SS_n_transitions_normal = (1'b1 => 0 [*12] => 1);
            bins SS_n_transitions_READ_DATA = (1'b1 => 0 [*22] => 1);
        }
        SS_ON_cp : coverpoint cov_seq_item.SS_n {
            bins SS_n_ON = (1'b1 => 1'b0 => 1'b0);
        }
        MOSI_valid_cp : coverpoint cov_seq_item.MOSI {
            bins MOSI_valid_transitions_WA = (1'b0[*3]);
            bins MOSI_valid_transitions_WD = (1'b0[*2] => 1'b1);
            bins MOSI_valid_transitions_RA = (1'b1[*2] => 1'b0);
            bins MOSI_valid_transitions_RD = (1'b1[*3]);
        }
        MOSI_SS_cross : cross MOSI_valid_cp , SS_ON_cp ;
        // these commented line represents the Illegal and legal bins of the mix between SS_n
time window and valid operations
        // option.cross_auto_bin_max = 0;

        // // Legal operation scenarios - Write operations with normal timing (13 cycles)
        // bins WRITE_ADDR_normal_timing = binsof(MOSI_valid_cp.MOSI_valid_transitions_WA)
&&
        // binsof(SS_n_cp.SS_n_transitions_normal);

```



```

//      bins WRITE_DATA_normal_timing = binsof(MOSI_valid_cp.MOSI_valid_transitions_WD)
&&
//
//      binsof(SS_n_cp.SS_n_transitions_normal);
//
//      // Legal operation scenarios - Read operations
//      bins READ_ADDR_normal_timing = binsof(MOSI_valid_cp.MOSI_valid_transitions_RA) &&
//      binsof(SS_n_cp.SS_n_transitions_normal);
//      bins READ_DATA_extended_timing = binsof(MOSI_valid_cp.MOSI_valid_transitions_RD)
&&
//
//      binsof(SS_n_cp.SS_n_transitions_READ_DATA);
//
//      // Exclude illegal combinations
//      ignore_bins WRITE_ADDR_wrong_timing =
binsof(MOSI_valid_cp.MOSI_valid_transitions_WA) &&
//      binsof(SS_n_cp.SS_n_transitions_READ_DATA);
//      ignore_bins WRITE_DATA_wrong_timing =
binsof(MOSI_valid_cp.MOSI_valid_transitions_WD) &&
//      binsof(SS_n_cp.SS_n_transitions_READ_DATA);
//      ignore_bins READ_ADDR_wrong_timing =
binsof(MOSI_valid_cp.MOSI_valid_transitions_RA) &&
//      binsof(SS_n_cp.SS_n_transitions_READ_DATA);
//      ignore_bins READ_DATA_wrong_timing =
binsof(MOSI_valid_cp.MOSI_valid_transitions_RD) &&
//      binsof(SS_n_cp.SS_n_transitions_normal);
//}
//}
endgroup

function new(string name = "SPI_coverage",uvm_component parent = null);
    super.new(name , parent);
    cvr_gp = new();
endfunction //new()

function void build_phase(uvm_phase phase);
    super.build_phase (phase);
    cov_export = new("cov_export",this);
    cov_fifo = new("cov_fifo",this);
endfunction
function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction
task run_phase (uvm_phase phase);
    super.run_phase (phase);
    forever begin
        cov_fifo.get(cov_seq_item);
        cvr_gp.sample();
    end
endtask
endclass
endpackage

```

12. Scoreboard

```

package SPI_scoreboard_pkg;
    import uvm_pkg::*;
    import SPI_shared_pkg::*;
    import SPI_sequence_item_pkg::*;
    `include "uvm_macros.svh"

```

```

class SPI_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(SPI_scoreboard)

    uvm_analysis_export #(SPI_sequence_item) sb_export;
    uvm_tlm_analysis_fifo #(SPI_sequence_item) sb_fifo;
    SPI_sequence_item sb_seq_item;

    int error_count , correct_count ;

    function new (string name = "SPI_scoreboard" , uvm_component parent = null);
        super.new(name,parent);
    endfunction
    function void build_phase (uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export",this);
        sb_fifo = new("sb_fifo",this);
    endfunction
    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction
    task run_phase (uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(sb_seq_item);
            // if(sb_seq_item.out != sb_seq_item.out_gld && sb_seq_item.leds !=
sb_seq_item.leds_gld ) begin
                if(sb_seq_item.rx_valid != sb_seq_item.rx_valid_gld || sb_seq_item.MISO !=
sb_seq_item.MISO_gld || sb_seq_item.rx_data !=sb_seq_item.rx_data_gld ) begin
                    // `uvm_error("run_phase",$sformatf("!!!!!!!!!!!!!!!!!!!!At time %0t: ERROR!!
transaction data is : %s",$realtime,sb_seq_item.convert2string_stimulus))
                    `uvm_error("run_phase",$sformatf("At time %0t: ERROR!! output data is :
%s",$realtime,sb_seq_item.convert2string))
                    error_count++;
                end
            else begin
                `uvm_info ("run_phase",$sformatf("At time %t: Correct transaction data is :
%s",$realtime,sb_seq_item.convert2string),UVM_HIGH);
                correct_count++;
            end
        end
    endtask
    function void report_phase (uvm_phase phase);
        super.report_phase(phase);
        `uvm_info ("report_phase",$sformatf("Correct_count = %d , Error_count =
%d",correct_count,error_count),UVM_MEDIUM);
    endfunction
endclass //SPI_scoreboard extends uvm_scoreboard

endpackage

```

13. Agent

```

package SPI_agent_pkg;
    import uvm_pkg::*;
    import SPI_shared_pkg::*;
    import SPI_sequence_item_pkg::*;

```

```

import SPI_sequencer_pkg::*;
import SPI_driver_pkg::*;
import SPI_config_pkg::*;
import SPI_monitor_pkg::*;

`include "uvm_macros.svh"
class SPI_agent extends uvm_agent;
    `uvm_component_utils(SPI_agent)

    SPI_config SPI_cfg;
    SPI_driver drv ;
    SPI_monitor mon;
    SPI_sequencer sqr ;
    uvm_analysis_port #(SPI_sequence_item) agt_ap ;

    function new (string name = "SPI_agent" , uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase (uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(SPI_config)::get(this,"","SPI_CFG",SPI_cfg)) begin
            `uvm_fatal("build_phase","unable to get config object")
        end
        if(SPI_cfg.is_active == UVM_ACTIVE) begin
            drv = SPI_driver::type_id::create("drv",this);
            sqr = SPI_sequencer::type_id::create("sqr",this);
        end
        mon = SPI_monitor::type_id::create("mon",this);
        agt_ap = new ("agt_ap",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        if(SPI_cfg.is_active == UVM_ACTIVE) begin
            drv.seq_item_port.connect(sqr.seq_item_export);
            drv.vif_cfg = SPI_cfg.SPI_vif_cfg;
        end
        mon.vif_cfg = SPI_cfg.SPI_vif_cfg;
        mon.mon_ap.connect(agt_ap);
    endfunction
endclass //SPI_agent extends uvm_agent
endpackage

```



14. Driver

```
package SPI_driver_pkg ;
import uvm_pkg::*;
import SPI_shared_pkg::*;
import SPI_sequence_item_pkg::*;
`include "uvm_macros.svh"
class SPI_driver extends uvm_driver #(SPI_sequence_item);
    `uvm_component_utils(SPI_driver)
    virtual SPI_if vif_cfg;
    SPI_sequence_item stim_seq_item;

    function new (string name = "SPI_driver",uvm_component parent = null);
        super.new(name,parent);
    endfunction
    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            seq_item_port.get_next_item(stim_seq_item);
            vif_cfg.rst_n = stim_seq_item.rst_n;
            vif_cfg.MOSI =      stim_seq_item.MOSI;
            vif_cfg.SS_n =      stim_seq_item.SS_n;
            vif_cfg.tx_valid =  stim_seq_item.tx_valid;
            vif_cfg.tx_data =  stim_seq_item.tx_data;
            @(negedge vif_cfg.clk);
            seq_item_port.item_done();
            `uvm_info("run_phase",stim_seq_item.convert2string_stimulus(),UVM_MEDIUM)
        end
    endtask
endclass
endpackage
```

15. Monitor

```
package SPI_monitor_pkg;
import uvm_pkg::*;
import SPI_shared_pkg::*;
import SPI_sequence_item_pkg::*;
`include "uvm_macros.svh"
class SPI_monitor extends uvm_monitor ;
    `uvm_component_utils(SPI_monitor);

    virtual SPI_if vif_cfg;
    SPI_sequence_item mon_seq_item;

    uvm_analysis_port #(SPI_sequence_item) mon_ap;

    function new (string name = "SPI_monitor" , uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase (uvm_phase phase);
        super.build_phase(phase);
        mon_ap = new("mon_ap",this);
    endfunction

    task run_phase(uvm_phase phase);
```

```

    super.run_phase(phase);
    forever begin
        mon_seq_item = SPI_sequence_item::type_id::create("mon_seq_item");
        @(negedge vif_cfg.clk);

        mon_seq_item.MOSI = vif_cfg.MOSI;
        mon_seq_item.rst_n = vif_cfg.rst_n;
        mon_seq_item.SS_n = vif_cfg.SS_n;
        mon_seq_item.tx_valid = vif_cfg.tx_valid;
        mon_seq_item.tx_data = vif_cfg.tx_data;

        mon_seq_item.rx_valid = vif_cfg.rx_valid;
        mon_seq_item.MISO = vif_cfg.MISO;
        mon_seq_item.rx_data = vif_cfg.rx_data;

        mon_seq_item.rx_valid_gld = vif_cfg.rx_valid_gld;
        mon_seq_item.MISO_gld = vif_cfg.MISO_gld;
        mon_seq_item.rx_data_gld = vif_cfg.rx_data_gld;
        mon_ap.write(mon_seq_item);
        // `uvm_info("run_phase",mon_seq_item.convert2string_stimulus(),UVM_MEDIUM)
    end
endtask
endclass
endpackage

```

16. Sequencer

```

package SPI_sequencer_pkg;
import uvm_pkg::*;
import SPI_shared_pkg::*;
import SPI_sequence_item_pkg::*;
`include "uvm_macros.svh"
class SPI_sequencer extends uvm_sequencer #(SPI_sequence_item);
    `uvm_component_utils(SPI_sequencer)

    function new(string name = "SPI_sequencer" , uvm_component parent = null);
        super.new(name,parent);
    endfunction //new()
endclass //SPI_sequencer extends uvm_sequencer
endpackage

```

17. Assertions

```

import SPI_shared_pkg::*;

module SPI_assertions(SPI_if SPIIF);

// Reset
property P1;
    (@(posedge SPIIF.clk) !SPIIF.rst_n | => ((SPIIF.MISO == 1'b0) && (SPIIF.rx_valid == 1'b0) &&
    (SPIIF.rx_data == 1'b0) ));
endproperty
reset: assert property(P1);
cover property(P1);

```



```

property P2;
    @(posedge SPIIF.clk) disable iff (!SPIIF.rst_n)
        ((SPIIF.SS_n == 1'b0) ##1 (SPIIF.MOSI == 1'b0) ##1 (SPIIF.MOSI == 1'b0) ##1 (SPIIF.MOSI ==
1'b0))
        |-> ##10 (SPIIF.rx_valid == 1'b1) | => (SPIIF.SS_n == 1'b1)[->1];
endproperty

valid_command: assert property (P2);
cover property (P2);

property P3;
    disable iff (!SPIIF.rst_n)
        @(posedge SPIIF.clk)
        ((SPIIF.SS_n == 1'b0) ##1 (SPIIF.MOSI == 1'b0) ##1 (SPIIF.MOSI == 1'b0) ##1 (SPIIF.MOSI ==
1'b1))
        |-> ##10 (SPIIF.rx_valid == 1'b1) | => (SPIIF.SS_n == 1'b1)[->1]);
endproperty

valid_command_1: assert property (P3);
cover property (P3);

property P4;
    disable iff (!SPIIF.rst_n)
        @(posedge SPIIF.clk) ((SPIIF.SS_n == 1'b0) ##1 (SPIIF.MOSI == 1'b1) ##1 (SPIIF.MOSI == 1'b1) ##1
(SPIIF.MOSI == 1'b0))
        |-> ##10 (SPIIF.rx_valid == 1'b1) | => (SPIIF.SS_n == 1'b1)[->1]);
endproperty

valid_command_2: assert property (P4);
cover property (P4);

property P5;
    disable iff (!SPIIF.rst_n)
        @(posedge SPIIF.clk) ((SPIIF.SS_n == 1'b0) ##1 (SPIIF.MOSI == 1'b1) ##1 (SPIIF.MOSI == 1'b1) ##1
(SPIIF.MOSI == 1'b1))
        |-> ##10 (SPIIF.rx_valid == 1'b1) | => (SPIIF.SS_n == 1'b1)[->1]);
endproperty

valid_command_3: assert property (P5);
cover property (P5);

endmodule

```



17.1. Table of assertions

Feature	Assertion
reset is asserted, the outputs (MISO, rx_valid, and rx_data) are all low	$\neg \text{SPIIF.rst_n} \Rightarrow ((\text{SPIIF.MISO} == 1'b0) \ \&\& (\text{SPIIF.rx_valid} == 1'b0) \ \&\& (\text{SPIIF.rx_data} == 1'b0))$);
<i>IDLE</i> → <i>CHK_CMD</i> → <i>WRITE_ADD</i>	$((\text{SPIIF.SS_n} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b0))$ $\rightarrow \#\#10 (\text{SPIIF.rx_valid} == 1'b1) \rightarrow (\text{SPIIF.SS_n} == 1'b1)[\rightarrow 1]$);
<i>IDLE</i> → <i>CHK_CMD</i> → <i>WRITE_DATA</i>	$((\text{SPIIF.SS_n} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b1))$ $\rightarrow \#\#10 (\text{SPIIF.rx_valid} == 1'b1) \rightarrow (\text{SPIIF.SS_n} == 1'b1)[\rightarrow 1]$);
<i>IDLE</i> → <i>CHK_CMD</i> → <i>READ_ADD</i>	$((\text{SPIIF.SS_n} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b1) \ \#\#1 (\text{SPIIF.MOSI} == 1'b1) \ \#\#1 (\text{SPIIF.MOSI} == 1'b0))$ $\rightarrow \#\#10 (\text{SPIIF.rx_valid} == 1'b1) \rightarrow (\text{SPIIF.SS_n} == 1'b1)[\rightarrow 1]$);
<i>IDLE</i> → <i>CHK_CMD</i> → <i>READ_DATA</i>	$((\text{SPIIF.SS_n} == 1'b0) \ \#\#1 (\text{SPIIF.MOSI} == 1'b1) \ \#\#1 (\text{SPIIF.MOSI} == 1'b1) \ \#\#1 (\text{SPIIF.MOSI} == 1'b1))$ $\rightarrow \#\#10 (\text{SPIIF.rx_valid} == 1'b1) \rightarrow (\text{SPIIF.SS_n} == 1'b1)[\rightarrow 1]$); endproperty
<i>IDLE</i> → <i>CHK_CMD</i>	$(\text{cs} == \text{IDLE} \ \&\& (\text{SS_n} == 1'b0)) \Rightarrow (\text{cs} == \text{CHK_CMD})$);
<i>CHK_CMD</i> → <i>WRITE_DATA</i> or <i>WRITE_ADD</i>	$(\text{cs} == \text{CHK_CMD} \ \&\& (\text{SS_n} == 1'b0)) \Rightarrow ((\text{cs} == \text{WRITE}) \ \parallel (\text{cs} == \text{READ_ADD}) \ \parallel ((\text{cs} == \text{READ_DATA})))$);
<i>WRITE_DATA</i> → <i>IDLE</i>	$(\text{cs} == \text{WRITE} \ \&\& (\text{SS_n} == 1'b1)) \Rightarrow (\text{cs} == \text{IDLE})$);
<i>READ_ADD</i> → <i>IDLE</i>	$(\text{cs} == \text{READ_ADD} \ \&\& (\text{SS_n} == 1'b1)) \Rightarrow (\text{cs} == \text{IDLE})$);
<i>READ_DATA</i> → <i>IDLE</i>	$(\text{cs} == \text{READ_DATA} \ \&\& (\text{SS_n} == 1'b1)) \Rightarrow (\text{cs} == \text{IDLE})$);



18. Do file

```
1 vlib work
2 vlog -f src_files.list +cover=sbfec -coveropt 3 +define+SIM
3 vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -sv_seed 906904221 -l sim.log -coverage -assertdebug -do "coverage save -onexit coverage.ucdb"
4 #interesting seeds 376014177 906904221 17147361
5 #add wave /top/SPIIF/*
6 do wave.do
7 run -all
8 #default case of the branches
9 coverage exclude -src SPI_slave.sv -line 165 -code b
10 coverage exclude -src SPI_slave.sv -line 221 -code b
11 coverage exclude -src SPI_slave.sv -line 175 -code b
12 coverage exclude -src SPI_slave.sv -line 231 -code b
13
14 coverage save coverage_final.ucdb
15 coverage report -detail -cvg -directive -comments -file assertions_coverage_report.txt -instance=/top/SPI_SLAVE/SPI_ASSERT
16 coverage report -detail -cvg -comments -file functional_coverage_report.txt
17 coverage report -detail -code bcefs -file code_coverage_report.txt -du=SLAVE
18 coverage report -detail -file combined_coverage_summary.txt
19
20 #quit -f
```



19. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
SPI-01	Reset Functionality - All outputs must be reset to default values when rst_n is asserted low	Random reset generation with 10% probability of reset assertion using constraint c_rst_n in SPI_sequence_item	No specific coverage group for reset (covered implicitly through assertion coverage)	Scoreboard checks: rx_data=0 rx_valid=0 MISO=0. Assertions: P1 property validates reset behavior with cover property
SPI-02	SS_n Control - Slave select signal controls communication start/stop with proper timing	SS_n_timing controlled by post_randomize() function with 13-cycle normal operations and 23-cycle read_data operations	SS_n_transitions_normal (13 cycles) and SS_n_transitions_read_DATA (23 cycles)	behavior. State machine transitions from any state to IDLE when SS_n asserted
SPI-03	MOSI Command Detection - First 3 bits of MOSI must be valid command (000 001 110 111)	Constraint c_MOSI ensures first 3 bits are valid commands. Random MOSI_arr[11] generation with command constraint	MOSI_valid_cp coverpoint with transition bins: WA (000) WD (001) RA (110) RD (111).	DUT state machine CHK_CMD state decodes MOSI commands correctly. Golden model matches command interpretation.
SPI-04	Write Address Operation - Write address data when command is 000	Generated by c_MOSI constraint allowing 3'b000 command with randomized data bits	validates write address with 13-cycle timing, ignore_bins exclude invalid timing combinations	DUT WRITE state captures 10-bit data via shift register. Golden model SER_TO_PAR task validates data capture and rx_valid assertion
SPI-05	Write Data Operation - Write data when command is 001	Generated by c_MOSI constraint allowing 3'b001 command with randomized data bits	Cross coverage WRITE_DATA_normal_timing validates write data with 13-cycle timing, ignore_bins exclude 23-cycle timing	DUT WRITE state processes data identically to write address. Golden model validates data reception and rx_valid timing
SPI-06	Read Address Operation - Read address when command is 110	Generated by c_MOSI constraint allowing 3'b110 command. Sets f_read_addr flag in post_randomize()	Cross coverage READ_ADDR_normal_timing validates read address with 13-cycle timing, ignore_bins exclude 23-cycle timing	DUT READ_ADDR state sets received_address flag. Golden model sets is_address_received flag and validates state transition to enable subsequent read data
SPI-07	Read Data Operation - Read data when command is 111 after address received	Generated by c_MOSI constraint allowing 3'b111 command. Requires f_read_addr=1 to set f_read_data=1	Cross coverage READ_DATA_extended_timing validates read data with 23-cycle timing, ignore_bins exclude 13-cycle timing	DUT READ_DATA state uses tx_valid and tx_data for MISO output. Golden model read_DATA_TASK() validates parallel-to-serial conversion
SPI-08	Counter Management - 4-bit counter controls bit reception/transmission	Counter initialized to 10 in CHK_CMD state decrements during data phases. Reset sets counter=0	Counter operation covered implicitly through data reception/transmission coverage	DUT counter decrements from 10 to 0 during data capture. Golden model cycle_counter_SER_TO_PAR and cycle_counter_PAR_TO_SER validate timing
SPI-09	RX_Valid Signal - Assert when 10-bit data reception complete	rx_valid generated when counter reaches 0 in WRITE_READ_ADDR_READ_DATA states	Coverage implicit through cross coverage scenarios requiring rx_valid assertion	DUT asserts rx_valid when counter=0. Golden model SER_TO_PAR task asserts rx_valid_gld at cycle 8. Scoreboard compares rx_valid vs rx_valid_gld
SPI-10	RX_Data Capture - Serial to parallel conversion of MOSI data	MOSI data generated through randomized MOSI_arr[11] with valid command constraints	rx_data_msb_cp coverpoint covers MSB patterns (00 01 10 11) with transition coverage between all combinations	DUT shifts MOSI into rx_data[counter-1] position. Golden model SER_TO_PAR task implements identical shifting. Scoreboard compares rx_data vs rx_data_gld
SPI-11	TX_Valid Control - Enable MISO transmission during read data operations	tx_valid controlled by f_read_data flag in post_randomize(). Only high during read data operations	tx_valid usage covered through read data cross coverage scenarios	DUT READ_DATA state checks tx_valid to enable MISO output. Golden model read_DATA_TASK() uses tx_valid to trigger PAR_TO_SER task
SPI-12	TX_Data Processing - Parallel data transmitted serially on MISO	Random tx_data[7:0] generation in sequence_item. Used when tx_valid=1 during read data	tx_data coverage implicit through random value generation and read data scenarios	DUT outputs tx_data[counter-1] on MISO during READ_DATA. Golden model PAR_TO_SER task shifts Parallel_data_out for MISO_gld output
SPI-13	MISO Output Control - Serial data output during read operations	MISO driven by tx_data bits during READ_DATA state when tx_valid=1	MISO operation covered through read data cross coverage validation	DUT READ_DATA state drives MISO=tx_data[counter-1]. Golden model PAR_TO_SER drives MISO_gld=Parallel_data_out[7]. Scoreboard compares MISO vs MISO_gld
SPI-14	State Machine Transitions - Proper FSM behavior for all states	All state transitions triggered through SS_n and MOSI stimulus patterns from sequence generation	State coverage implicit through command and timing cross coverage scenarios	DUT FSM: IDLE->CHK_CMD->WRITE/READ_ADDR/READ_DATA->IDLE. Golden model implements identical state machine. Functional equivalence validated through output comparison
SPI-15	Address Reception Flag - Track address reception for read data enable	received_address flag set during READ_ADDR completion enables subsequent READ_DATA operation	Flag behavior covered through read address followed by read data cross coverage scenarios	DUT sets received_address=1 after READ_ADDR completion. Golden model sets is_address_received=1. Flag enables READ_DATA state entry in subsequent transactions
SPI-16	Cycle Timing Compliance - 13 cycles for normal operations 23 cycles for read data	Timing controlled by post_randomize() function: counter_SS_n tracks cycles and controls SS_n assertion	SS_n_cp coverpoint specifically validates 13-cycle (SS_n_transitions_normal) and 23-cycle (SS_n_transitions_read_DATA) patterns	DUT expects SS_n_low for data duration then high for 1 cycle. Golden model operates on same timing assumptions. Cross coverage validates legal timing-command combinations
SPI-17	Command Validation - Only valid commands processed illegal commands ignored	Constraint c_MOSI restricts to valid commands: 000(write_addr) 001(write_data) 110(read_addr) 111(read_data)	Cross coverage bins validate only legal command-timing combinations with ignore_bins for illegal scenarios	DUT CHK_CMD state routes to appropriate data handling state based on valid commands. Golden model implements identical command-decode logic.
SPI-18	Data Integrity - Complete 10-bit data reception before rx_valid assertion	Counter mechanism ensures all 10 bits captured before rx_valid=1. Data shifted serially into rx_data register	Data integrity covered through MSB pattern coverage and transition patterns in rx_data_msb_cp	DUT shifts MOSI->rx_data[counter-1] for 10 cycles before rx_valid. Golden model SER_TO_PAR implements identical shift register. Bit-by-bit validation through scoreboard
SPI-19	Concurrent Operation Handling - Proper behavior during overlapping read data phases	tx_valid can assert during READ_DATA state creating concurrent rx/tx operation scenarios	Concurrent operations covered through read data timing scenarios where tx_valid assertion during data reception	DUT READ_DATA handles both MOSI capture and tx_data transmission based on tx_valid. Golden model read_DATA_TASK manages concurrent operations identically
SPI-20	Reset Recovery - Proper initialization after reset deassertion	Reset sequence generates rst_n=0 followed by rst_n=1 transitions. Corner sequence provides reset scenarios	Reset coverage through assertion property P1 with cover property validation	DUT initializes all registers to 0 state after reset. Golden model identical reset behavior. Scoreboard validates proper reset state: rx_data=0 rx_valid=0 MISO=0
SPI-21	Interface Signal Compliance - All signals operate within interface modport definitions	SPI_if interface with separate DUT and gld modports. Testbench uses signals through virtual interface	Interface compliance validated through compilation and signal access patterns	DUT connects through SPI_if gld modport. Golden model through SPI_if gld modport. Interface ensures proper signal direction and access control



20. Coverage

- You can access the 3 coverages report of SPI_Slave through this link :

Coverage reports

	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
 jvnm_pkg:svvm_reg_map:do_write/#ubk#215181159+1731/immed...	Immediate	SVA	on	0	0	0	-	-	-	-	off	assert (\$cast(seq.o))	✓
 jvnm_pkg:svvm_reg_map:do_read/#ubk#215181159+1771/immed...	Immediate	SVA	on	0	0	0	-	-	-	-	off	assert (\$cast(seq.o))	✗
 jvnm_sequence_pkg:ALSU_main_sequence:body/#ubk#37877255...	Immediate	SVA	on	0	1000	0	-	-	-	-	off	assert (randomize(...))	✓
 jvnm_sequence_pkg:ALSU_main_sequence:body/#ubk#37877255...	Immediate	SVA	on	0	3000	0	-	-	-	-	off	assert (randomize(...))	✓
 jvnm_sequence_pkg:ALSU_main_sequence:body/#ubk#37877255...	Immediate	SVA	on	0	1000	0	-	-	-	-	off	assert (randomize(...))	✓
 /top/DUT/ALSU_ASSERT/a_reset	Concurrent	SVA	on	0	4555	0	08	1608	179 ns	4555	off	assert (@(posedge ALSUIF.clk) (AL...	✓
 /top/DUT/ALSU_ASSERT/a_invalid_outbypass	Concurrent	SVA	on	0	321	1	808	2408	685 ns	668	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_invalid_leds	Concurrent	SVA	on	0	1539	1	808	2408	685 ns	2358	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_bypassA	Concurrent	SVA	on	0	748	0	08	2408	2907 ns	1236	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_bypassB	Concurrent	SVA	on	0	741	0	08	2408	3461 ns	1249	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_both_bypass_priority_A	Concurrent	SVA	on	0	667	0	08	2408	2607 ns	1076	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_both_bypass_priority_B	Concurrent	SVA	on	0	0	0	08	08	0 ns	0	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_OR	Concurrent	SVA	on	0	82	0	08	1608	43 ns	102	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_XOR	Concurrent	SVA	on	0	96	0	08	1608	27 ns	140	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_ADD_cn_off	Concurrent	SVA	on	0	0	0	08	08	0 ns	0	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_ADD_cn_on	Concurrent	SVA	on	0	35	0	08	1608	1279 ns	83	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_MULT	Concurrent	SVA	on	0	34	0	08	1608	1557 ns	83	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_A_or	Concurrent	SVA	on	0	31	0	08	808	7683 ns	37	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_A_XOR	Concurrent	SVA	on	0	71	0	08	2408	1493 ns	192	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_B_or	Concurrent	SVA	on	0	29	0	08	808	4679 ns	37	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_B_XOR	Concurrent	SVA	on	0	52	0	08	2408	1711 ns	170	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_OR_both_periority_A	Concurrent	SVA	on	0	27	1	808	808	2451 ns	43	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_XOR_both_periority_A	Concurrent	SVA	on	0	30	0	08	808	2367 ns	44	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_OR_both_periority_B	Concurrent	SVA	on	0	0	0	08	08	0 ns	0	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_RED_XOR_both_periority_B	Concurrent	SVA	on	0	0	0	08	08	0 ns	0	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_left_shift	Concurrent	SVA	on	0	9	0	08	1608	1145 ns	50	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_right_shift	Concurrent	SVA	on	0	19	0	08	1608	219 ns	80	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_left_rotate	Concurrent	SVA	on	0	20	0	08	1608	1153 ns	45	off	assert (@(posedge ALSUIF.clk) dis...	✓
 /top/DUT/ALSU_ASSERT/a_right_rotate	Concurrent	SVA	on	0	27	0	08	1608	1839 ns	79	off	assert (@(posedge ALSUIF.clk) dis...	✓

21. Questa Snaps

Path	Coverage	Count	Target	Pass	Fail	Warning	Info	Auto
/SPI_coverage_pkg/SPI_coverage	100.00%	100	100.00%	✓				auto(0)
TYPE cvr_gp	100.00%	100	100.00%	✓				
CVP cvr_gp::rx_data_msb_cp	100.00%	100	100.00%	✓				
CVP cvr_gp::SS_n_cp	100.00%	100	100.00%	✓				
CVP cvr_gp::SS_ON_cp	100.00%	100	100.00%	✓				
CVP cvr_gp::MOSI_valid_cp	100.00%	100	100.00%	✓				
CROSS cvr_gp::MOSI_SS_cross	100.00%	100	100.00%	✓				
INST /SPI_coverage_pkg::SPI_coverage::cvr_gp	100.00%	100	100.00%	✓				
CVP rx_data_msb_cp	100.00%	100	100.00%	✓				
bin msb_00	14475	1	100.00%	✓				
bin msb_01	12828	1	100.00%	✓				
bin msb_10	11807	1	100.00%	✓				
bin msb_11	11896	1	100.00%	✓				
bin msb_transitions	3820	1	100.00%	✓				
CVP SS_n_cp	100.00%	100	100.00%	✓				
bin SS_n_transitions_normal	3384	1	100.00%	✓				
bin SS_n_transitions_READ_DATA	244	1	100.00%	✓				
CVP SS_ON_cp	100.00%	100	100.00%	✓				
bin SS_n_ON	3749	1	100.00%	✓				
CVP MOSI_valid_cp	100.00%	100	100.00%	✓				
bin MOSI_valid_transitions_WA	12562	1	100.00%	✓				
bin MOSI_valid_transitions_WD	7079	1	100.00%	✓				
bin MOSI_valid_transitions_RA	4935	1	100.00%	✓				
bin MOSI_valid_transitions_RD	3949	1	100.00%	✓				
CROSS MOSI_SS_cross	100.00%	100	100.00%	✓				
bin <MOSI_valid_transitions_RD,SS_n_ON>	411	1	100.00%	✓				
bin <MOSI_valid_transitions_RA,SS_n_ON>	409	1	100.00%	✓				
bin <MOSI_valid_transitions_WD,SS_n_ON>	519	1	100.00%	✓				
bin <MOSI_valid_transitions_WA,SS_n_ON>	504	1	100.00%	✓				

